

Architecture and SOTA Mapping Report

Two simplified heuristic approaches for Maximum Agreement Forest

Prepared from the supplied implementations and the supplied SOTA analysis document.

Naming convention used in this report:

Architecture A – Candidate-pool / packing solver

Architecture B – Region-based local-search solver

The report intentionally uses architecture labels rather than implementation file names.

Scope

This report explains two standalone heuristic architectures for the Maximum Agreement Forest (MAF) problem. It focuses on how an input instance is transformed into a good forest, what subcomponents each architecture contains, and how each architecture maps to the SOTA categories in the supplied analysis document. The goal is explanatory clarity rather than proving optimality or reproducing all contest-level tuning.

Executive summary

Architecture	Core search idea	Closest SOTA family	Explainability
A – Candidate-pool / packing	Build a large menu of valid components, then greedily pack a compatible subset.	Kernelization plus branch-and-price-like column generation, with greedy repair.	High: the pipeline is global and linear.
B – Region-based local search	Start with a valid forest, then repeatedly replace small regions with better local covers.	Large-neighborhood search after reduction, with local exact-cover subproblems.	Medium-high: the local-search idea is clear, but there are more region limits.

The main conceptual difference is this: Architecture A tries to build the answer from a large menu of valid components, while Architecture B starts with an answer and repeatedly tries to merge or rearrange small neighborhoods into fewer components.

SOTA framework used for the mapping

The supplied SOTA document defines the MAF objective as finding a minimum partition of leaf labels such that the induced subtrees are isomorphic and edge-disjoint across the input trees. It groups current approaches into kernelization/reduction, greedy approximation, local search and metaheuristics, LP/ILP and branch-and-price, pruned branch-and-bound, and hitting-set reductions [1].

SOTA family	Meaning in the document	What this report looks for
Kernelization and reduction	Shrink the instance using subtree, chain, or cluster rules before search.	Common-subtree contraction and atomization of regions already identical in both trees.
Greedy approximation	Fast constructive decisions that produce a usable solution quickly.	Greedy ordering, greedy packing, or greedy merge choices.
Local search and metaheuristics	Improve an incumbent forest by modifying neighborhoods of the solution.	Region selection, destroy-and-repair, exact local replacement, or repeated repair loops.
LP/ILP and branch-and-price	Use components/columns and pricing; exact solvers may be time-limited.	Validated component candidates as columns, plus restricted packing of compatible columns.
Pruned branch-and-bound	Explore only promising branches, often inside decomposed subproblems.	Bounded exact-cover search on small conflict components or small regions.
3-Hitting Set reductions	Represent local conflicts as sets of cuts to hit.	Set-cover-like or conflict-cover-like subproblems; not necessarily the full formal reduction.

Architecture in MAF terms

Both architectures solve the same pipeline problem. They differ mainly in how they search for good components after reduction.

```
Two rooted trees
-> parse Newick
-> compress identical common subtrees into atoms
-> search for valid agreement-forest components
-> choose compatible components
-> expand atoms back to original labels
-> print one Newick component per line
```

Shared core idea: component masks

Both architectures represent a possible agreement component as a bit mask. After reduction, each atom receives a bit position. A component is stored as one integer whose set bits indicate the atoms included in that component. This makes overlap tests, unions, containment tests, and component sizes very cheap.

Reduced atom	Bit position	Example component $\{A, C\}$
A	0	1
B	1	0
C	2	1
D	3	0

For example, if atom 0 is A and atom 2 is C , then the component $\{A, C\}$ is represented by setting bits 0 and 2. A forest is then a list of such masks. A forest like $\{A, B\}, \{C\}, \{D\}$ becomes conceptually equivalent to $[0011, 0100, 1000]$.

Operation	Bit-mask expression	Why it is useful
Do two components overlap?	<code>mask1 & mask2</code>	A nonzero result means they share at least one atom.
Merge two proposed components	<code>mask1 mask2</code>	Builds a union mask without copying sets.
Remove atoms from a component	<code>mask1 & ~mask2</code>	Keeps atoms in the first mask that are not in the second.
Test containment	<code>mask1 & mask2 == mask1</code>	Checks whether all atoms of <code>mask1</code> lie inside <code>mask2</code> .
Count component size	<code>mask.bit_count()</code>	Counts the number of reduced atoms in the component.

Shared data objects

Object	Meaning
Node	One rooted tree node. It stores children, a leaf/component mask, parent information, node id, and edge id.
Annotation	Cached node information: descendant leaf set, canonical rooted shape, and leaf count.
ReducedInstance	The compressed two-tree instance after exact common-subtree reduction. Its atoms may represent original leaves or already-agreeing subtrees.
ValidationInfo	Evidence that a mask is a legal component: matching restricted topology plus the tree edges used by that component.
Candidate	A valid possible component with its mask, weight, size, and consumed edges.
CandidatePool	A collection of valid candidates plus a validation cache. In the simplified implementations, it is the place where proposals are filtered into legal components.

The central safety check

The most important method conceptually is `CandidatePool.validate_mask(mask)`. It asks: if exactly these atoms are taken, do they induce the same rooted shape in Tree 1 and Tree 2? If yes, the mask is a legal agreement component. The validator also records which tree edges are used, because selected components must be disjoint not only in atoms but also in their consumed edges.

```
many heuristic mask proposals
-> strict topology validation
-> valid candidates with edge-use information
-> compatibility checks during packing or local cover
```

Output behavior and safety fallback

Both architectures ultimately convert reduced atom masks back to original labels and print one Newick component per line, using the first input tree as the shape reference for rendering. Before output, the final components are validated. If validation fails, the safe fallback is the forest of all singletons. That fallback is poor for score, but it is always valid.

Architecture A – Candidate-pool / packing solver

High-level idea

Architecture A is a global candidate-market solver. It generates many possible components, validates them, greedily packs a compatible subset, and then applies a small merge repair. The mental model is a marketplace of possible components: each valid component is a candidate column, and the solver tries to buy a non-conflicting set of large useful chunks.

```
Candidate 1: {A, B}
Candidate 2: {C, D, E}
Candidate 3: {A, C}
Candidate 4: {F, G}
...
Validation keeps only the candidates whose restricted topology agrees in both trees.
```

Large components are valuable because they reduce the final number of components. A component of size k has weight $k - 1$, because it saves $k - 1$ components compared with leaving the k atoms as singletons.

Instance-to-solution pipeline

Stage	What happens	Motivation
1. Parse and reduce	The two Newick trees are parsed and exact common subtrees are contracted into atoms.	Already-agreeing regions do not need to be searched again. This is the kernelization-like part.
2. Generate many candidate masks	Pair, interval, subtree/intersection, and union generators propose masks that might form components.	Broad generation gives the packer a useful menu, while validation prevents bad guesses from entering the solution.
3. Validate candidates	Every proposed mask is checked by restricted topology equality in the two trees.	This is the main correctness filter. Generation can be heuristic because validation is strict.
4. Greedily pack candidates	The solver chooses non-conflicting candidates using a few simple orderings.	The packing tries to cover many atoms in large chunks while respecting atom and edge disjointness.
5. Add singletons	Uncovered atoms are added as singleton components.	This guarantees a complete forest even if the candidate pool misses some atoms.
6. Small repair pass	The solver tries conservative pair merges among current components.	If two current components can become one valid component without edge conflicts, the component count improves.

Candidate generators and why they exist

Generator	Intuition	Why this choice is reasonable
Pair masks	Leaves close in DFS order or under small rooted envelopes are likely to form useful small components.	Pairs are cheap to test and provide a fallback structure on large instances.
Interval masks	Contiguous or compact spans in tree traversal can correspond to common chains or local regions.	They cover chain-like structure without implementing a full chain-reduction kernel.
Subtree and intersection masks	Small subtrees from either input tree and their overlaps are structurally plausible components.	Agreement components often align with clades or intersections of clades.
Candidate unions	Two already-valid chunks that lie near each other may merge into a larger valid chunk.	This is a simple pricing-like expansion step: grow useful columns from smaller columns.

Packing and conflict model

Two candidates conflict if they use the same atom, reuse a Tree 1 edge, or reuse a Tree 2 edge. The greedy packer therefore maintains used atoms and used edges while scanning candidates in several motivated orders: high saving per edge, large components first, and low edge cost. These are heuristic choices, but their motivation is simple: prefer components that save many singleton pieces while consuming few scarce tree edges.

SOTA mapping

Only strong and strong-to-medium correspondences are shown.

SOTA category	Relevance	How it fits
Kernelization and reduction	Strong	The instance is first simplified by contracting exact common rooted subtrees into atoms. This directly matches the SOTA idea that preprocessing can preserve the essential search problem while reducing the instance.
LP/ILP and branch-and-price	Strong heuristic analogue	Valid components behave like columns. Candidate generation is a limited pricing-like step, and packing the compatible subset is the restricted master-problem analogue. The implementation is heuristic and does not solve an LP.
Greedy approximation / constructive heuristics	Strong-to-medium	The main selection phase is greedy and quickly constructs a full forest. It uses approximation-style intuition, but without a formal approximation guarantee.

Explainability and arbitrary choices

This is the easier of the two complete architectures to explain. The main arbitrary choices are the generator limits, interval sizes, candidate caps, and greedy ordering formulas. These choices are performance knobs, but they all serve one understandable goal: build a useful but bounded candidate menu.

Architecture B – Region-based local-search solver

High-level idea

Architecture B is an incumbent-driven local-search solver. It first builds a reasonable complete forest, then repeatedly opens small tree-shaped regions and solves each region as a small exact-

cover problem. Its philosophy is: do not solve the whole instance globally; improve one bounded neighborhood of the current solution at a time.

```
current forest
-> choose a small region of components
-> generate local replacement candidates
-> exactly cover the same atoms with fewer components
-> accept if improved
-> repeat
```

Instance-to-solution pipeline

Stage	What happens	Motivation
1. Parse and reduce	The input trees are parsed and exact common subtrees are contracted into atoms.	As in Architecture A, obvious agreement is compressed before search.
2. Build a modest initial pool	The solver creates candidates from small subtrees, intersections, and simple consensus-like structures.	The goal is not a huge global market, but a reasonable initial forest.
3. Track the current forest	Each current component receives an id, and each atom is mapped to the component that owns it.	Local search needs to know which components are inside each region.
4. Pick local regions	A region is a small set of current components lying under some node or local area of either tree.	Tree-shaped regions are likely places where several components can be merged or rearranged.
5. Generate local replacements	The region receives candidates from existing components, known good chunks, unions, tree-node cuts, and singletons.	Existing components and singletons preserve feasibility; new unions and cuts offer improvement.
6. Solve exact cover inside the region	A bounded search covers exactly the same atoms with valid non-conflicting local components.	Because the region is small, exact local search is feasible and can repair greedy mistakes.
7. Repeat for several rounds	Accepted improvements update the forest; the solver continues until no region improves or the round limit is reached.	A good forest emerges through repeated local recombination.

Local region example

Suppose the current forest contains $\{A\}$, $\{B\}$, and $\{C, D\}$. If a tree node contains A , B , C , and D , the corresponding region may consist of those three current components. The local optimizer asks whether the three components can be replaced by two or one valid components, such as $\{A, B\}$, $\{C, D\}$ or $\{A, B, C, D\}$. If the replacement is valid and uses no forbidden outside edges, the forest improves.

Local replacement candidates

Candidate source	Meaning	Why it is included
Existing components	The old components inside the region.	They keep the local problem feasible if no improvement exists.
Strong global candidates	Previously validated useful chunks that lie fully inside the region.	They reuse information found during initial construction.
Unions of current components	Several current pieces merged into one proposed component.	This is the main way to reduce component count.
Tree-node cuts inside the region	Natural local subtrees or separators restricted to the region.	Tree structure often suggests valid replacement components.
Singletons	One atom by itself.	They guarantee fallback coverage and prevent the exact cover from getting stuck.

Exact cover inside a region

The key step is a bounded exact-cover search. For one selected region, the solver chooses non-overlapping local components that cover exactly the same atoms as before, do not reuse forbidden outside edges, and use fewer components than the old region. This is why Architecture B maps naturally to large-neighborhood search: it temporarily destroys a small part of the incumbent forest and rebuilds it better.

SOTA mapping

Only strong and strong-to-medium correspondences are shown.

SOTA category	Relevance	How it fits
Local search and large-neighborhood search	Strong	This is the defining mechanism: after an initial feasible forest exists, the solver repeatedly selects bounded neighborhoods and tries to rebuild them with fewer components.
Pruned branch-and-bound	Strong-to-medium	The regional exact-cover subproblem is solved by bounded search with pruning and a target better than the current component count. It is local rather than global branch-and-bound.
Set-cover / hitting-set style formulations	Strong-to-medium	Inside a region, the solver chooses valid components that cover a fixed set of atoms while respecting conflicts. This is set-cover-like, though not the full global 3-Hitting Set reduction.
Kernelization and reduction	Strong-to-medium	The architecture still begins with exact common-subtree contraction, but reduction is a supporting stage rather than the distinctive idea.

Explainability and arbitrary choices

This architecture is more different from Architecture A, but it has more bounded-search knobs: region size, maximum number of components per region, number of improvement rounds, local candidate caps, and search limits. These choices are still motivated by one clear need: exact local cover is useful only while the neighborhood remains small enough to search quickly.

Main conceptual difference

Aspect	Architecture A	Architecture B
Search style	Global candidate packing.	Local improvement of an incumbent forest.
Main object	A large candidate pool.	The current forest and its regions.
Key question	Which valid components should be selected globally?	Can this small region be replaced by fewer components?
Strength	Simple, broad, fast candidate selection.	Can repair greedy mistakes locally.
Weakness	Greedy packing can miss good combinations.	Depends on finding useful regions and keeping them small.
Analogy	Shopping from a big menu.	Editing one paragraph at a time.

Tiny comparison example

Suppose the best forest should contain $\{A, B, C\}$, $\{D, E\}$, and $\{F\}$. Architecture A tries to generate $\{A, B, C\}$ and $\{D, E\}$ as candidates early, validate them, and pack them globally. Architecture B might start worse, for example with $\{A, B\}$, $\{C\}$, $\{D, E\}$, $\{F\}$. It then notices a region containing $\{A, B\}$ and $\{C\}$ and asks whether it can replace them with $\{A, B, C\}$. If yes, it improves to the same better forest.

Focused SOTA mapping summary

Architecture	Primary mapping	Secondary mappings	How direct is the mapping?
A – Candidate-pool / packing	Branch-and-price-like column generation.	Kernelization, greedy construction, small repair.	Direct as a heuristic analogue: columns are generated and packed, but no LP solver is used.
B – Region-based local search	Large-neighborhood search.	Pruned local exact cover, set-cover-like regional replacement, kernelization.	Direct for LNS: it literally improves bounded neighborhoods of an incumbent forest.

Instance transformation diagrams

Architecture A – candidate-pool / packing

Input trees

- > parse and annotate
- > exact common-subtree contraction
- > reduced atom instance
- > pair, interval, subtree, intersection, and union masks
- > validation by restricted signatures
- > global greedy packing under atom and edge conflicts
- > singleton completion and pair-merge repair
- > decoded agreement forest

Architecture B – region-based local search

Input trees

- > parse and annotate
- > exact common-subtree contraction
- > modest initial candidate pool
- > initial packed forest
- > select bounded regions of the current forest
- > generate local replacement candidates
- > solve exact local cover
- > apply improving replacements until stalled or out of rounds
- > decoded agreement forest

Conclusions

- Both architectures share the same MAF safety invariant: a proposed component is not trusted until the restricted topology matches in both trees and its atom/edge conflicts are respected.
- Architecture A is the clearest global construction architecture: generate many candidate components, validate them, and pack a compatible subset.
- Architecture B is more different from Architecture A because it treats the current forest itself as the search object and improves bounded neighborhoods by exact cover.
- The key explanatory contrast is menu versus editing: Architecture A selects from a large menu of valid chunks, while Architecture B edits one local region of the current forest at a time.

References

- [1] Supplied SOTA analysis document for PACE 2026 MAF heuristic approaches, pp. 1–3. Used for the taxonomy of kernelization/reduction, greedy approximation, local search, LP/ILP and branch-and-price, pruned branch-and-bound, and hitting-set reductions.
- [2] Supplied source implementations for the two retained architectures. The report intentionally uses architecture labels instead of file names, per instruction.